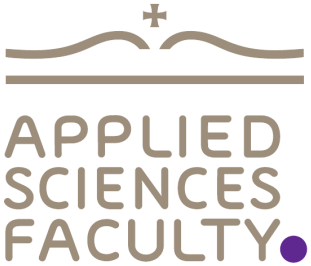
C++ — лекція 26. Rvalue-посилання та операція переміщення



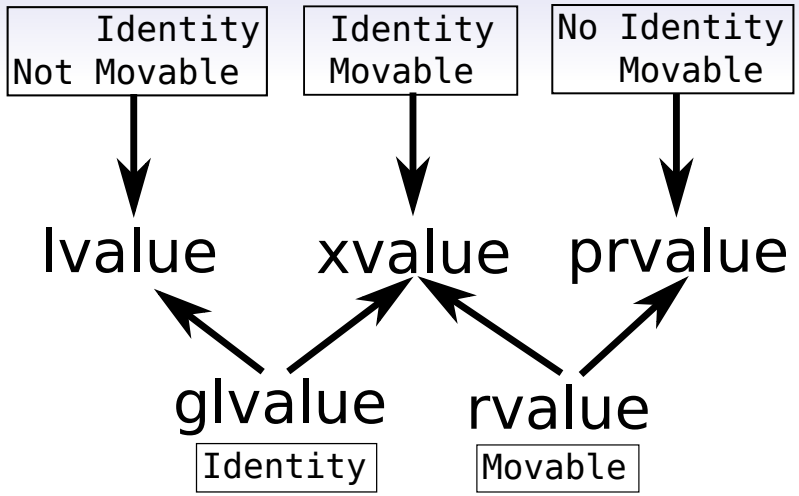


©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Підтримка можливості переміщення — rvalue-посилання та move-семантика

- Прозора та сумісна підтримка можливості переміщення із тимчасових об'єктів вимагає доволі складного механізму. Для його підтримки було введено ряд нових понять. Розглянемо їх.
- Кожен об'єкт може володіти[1] двома властивостями, важливими в даній ситуації:
- Перша — він може мати **ідентичність** (identity), тобто програма володіє іменем об'єкту чи вказівником або посиланням на нього, може перевірити, чи два об'єкти співпадають і працювати із таким об'єктом.
- Друга — здатність бути переміщеним (movable), виражає дозвіл переміщення стану із об'єкту.
- Із чотирьох можливих комбінацій використовуються три — об'єкти без ідентичності, із яких не можна переміщати, позбавлені сенсу.

[1] А відповідно, може і не володіти.



- lvalue — має ідентичність, не може бути переміщеним.
- rvalue — будь що, з чого можна переміщати[2].
- xvalue — "eXpiring value"[3], об'єкт із ідентичністю, час життя якого збігає.
- prvalue — "pure rvalue", таке rvalue, що не є xvalue.
- glvalue — узагальнене (generalized) lvalue: lvalue або xvalue.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/value_category` та `https://blog.panicsoftware.com/value-categories-gl-l-x-r-prvalues/`.

[2] Літерали фундаментальних типів, крім стрічкових літералів, теж rvalue, але для них переміщення не є окремою операцією, вони тільки копіюються. Переміщення користувацьких літералів має сенс, але еквівалентне переміщенню тимчасових значень.

[3] Страуструп жартує, що може розшифровуватися як "eXtraordinary" чи "eXperts only".

lvalue – nonmovable, has identity

Приклади[4]

- Ім'я змінної, функції чи члена даних.
- Аналогічно, звертання до них за вказівником.
- Виклик функції, результатом якої є lvalue reference, наприклад ++i, a+=b, чи vec[i], де vec – змінна типу vector<T> чи аналогічного. Стосується і операцій над вбудованими типами.
- Також, lvalue є стрічковий літерал. Стверджують[5], що це спадщина C – літерал являє собою масив, який в C – завжди lvalue.
- Навіть якщо тип змінної – rvalue- посилання, сама змінна – lvalue!

Риси:

- Можна брати адресу lvalue.
- Не константній lvalue можуть знаходитися ліворуч від оператора присвоєння та використовуватися для ініціалізації lvalue-посилань, фактично – створення нового імені для того ж об'єкту.
- Див. також glvalue.

[4] Список далеко не вичерпний! Див. http://en.cppreference.com/w/cpp/language/value_category.

[5] Див. https://stackoverflow.com/questions/10004511/why-are-string-literals-l-value-while-all-other-literals-are-r-value.

prvalue – movable, no identity

Приклади

- Літерали, крім стрічкових.
- Виклики функцій та вбудованих операторів, результати яких не є lvalue-посиланнями: a+b, i++.
- Взяття адреси: &a.
- Вказівник this.
- Лямбда-функції.

Риси:

- Не може бути поліморфним – завжди має конкретний тип.
- Не може мати cv-кваліфікаторів.
- Див. також rvalue.

xvalue – movable, has identity

Приклади:

- Виклик, результатом якого є rvalue-посилання, наприклад std::move(x) чи відповідні приведення типів, `static_cast<char&&>(x)`.
- a[n] для вбудованих масивів, a.m (не статичний, не посилання),

Риси:

- Див. rvalue і glvalue.
- Зокрема, може бути прив'язаним до rvalue-reference (див. далі), але може бути поліморфним.

glvalue – lvalue або xvalue

Риси:

- glvalue може неявно приводитися до prvalue, як lvalue -> rvalue, чи як масив до вказівника, функція до вказівника.
- Може бути поліморфним – фактичний тип об'єкту, якого glvalue стосується, може відрізнятися від типу виразу.

rvalue – prvalue або xvalue

Риси:

- Адресу брати не можна.
- Не може бути ліворуч від оператора присвоєння.
- Може бути використано для ініціалізації константного lvalue-посилання чи **rvalue-посилання**, тоді життя rvalue продовжується на час життя відповідного посилання.
- Однак, якщо існує два перевантаження функції, одне із яких приймає const lvalue-reference, а інше – rvalue-reference, буде вибрано друге. Це дає можливість виконувати переміщення там, де воно потрібне, наприклад, якщо є і конструктор копіювання і конструктор переміщення, вибрати переміщення.

rvalue-посилання

- Можна сказати, що lvalue-посилання дозволяє змінювати об'єкт, константне lvalue посилання змінювати об'єкт не дозволяє. rvalue-посилання посилається на об'єкт, стан якого ми не потребуємо після звертання.
- Відповідно, константні rvalue-посилання позбавлені особливого сенсу. Хоча, екзотичні використання для них є.
- lvalue-посилання можна прив'язати до (змусити посилатися на) lvalue[6]
- rvalue-посилання, навпаки, може бути прив'язане лише до rvalue.
- Позначається rvalue-посилання двома символами &:

```
int fn1() {     return 7;     }

int&& lrx = fn1();
```

[6] Із важливим винятком, що const-lvalue-reference може посилатися на rvalue.

Перевантаження для rvalue-посилань

- Перевантаження може вибирати між lvalue-reference і rvalue-reference! Таким чином, для ситуацій, коли вихідний об'єкт більше не потрібен (точніше, більше не потрібен його стан), можна використати більш ефективні алгоритми. Класичний приклад — конструктори копіювання та оператори присвоєння. (Див. далі.)
- Щоб перевантажити метод по тому, застосовано його до lvalue чи rvalue, можна скористатися таким синтаксисом:

```
struct test{
  void f() &{ std::cout << "lvalue object\n"; }
  void f() &&{ std::cout << "rvalue object\n"; }
};
```

- Приклад:

```
struct C1 {    double x;          };
int fn1() {    return 7;     }
C1 fnC()  {    C1 v;     v.x = 5;     return v;     }

    int xv=11;
    int& rxv = xv;
    // int& rx = fn1(); // <-- Error
    // C1 &rc = fnC();  // <-- Error
    const int& rx = fn1();
    const C1 &rc = fnC();

    int&& lrx = fn1();
    C1 &&lrc = fnC();

    // int&& lrxv = xv; // <-- Error
    int&& lrxv = move(xv);
```

- Якщо точно відомо, що lvalue більше не знадобиться, можна примусово зробити із нього rvalue-посилання, привівши до відповідного типу.
- Для цього існує спеціальна функція std::move() із <utility> – див. приклад.

std::move()

- Фактично, move() здійснює перетворення, static_cast<> до T&&. Тому назва у неї достатньо невдала. Страструп каже[7], що краще б було назвати rval(), але пізно.
- Можлива реалізація (constexpt з'явилося в C++14):

```
template<typename T>
constexpr typename std::remove_reference<T>::type&&
    move(T&& t) noexcept
    {
        return static_cast<typename
                    std :: remove_reference<T>::type&&>(t);
    }
```

- remove_reference<>::type "повертає" T, якщо її аргументом буде T, T& чи T&& — "здирає" із типу аргументу шаблону всі посилання. Див. також наступний слайд.
- Есей на протилежну тему: "How do I make an expression non-movable? What's the opposite of std::move?" – константні об'єкти не можна переміщати.

[7] Стор. 195 4-го видання.

Колапс посилань[8] — посилання на посилання

- Посилання на посилання створити неможливо. Однак, трапляються ситуації, де компілятору доводиться мати справу із такими спробами. Наприклад:

```
void fnarg(double& x) {   .....   }
    ..................

    double k1=5;
    double &rk1 = k1;
    fnarg(rk1);
```

Функція приймає посилання і їй передається посилання. Таке подвійне & і & дає просто &.

- Більш складний випадок – із шаблонами.

```
template <typename T>
void tmpl_fnarg(T& x) {   .....   }
    ..................
    tmpl_fnarg(rk1);
```

Тип T все рівно буде double — **НЕ** double&.

[8] Reference collapsing

- Ще один спосіб побачити колапс посилань — скористатися синонімами типів:

```
using r_to_i = int&;
using r_to_r_to_i = r_to_i&; //! Just int& ––– collapsed
```

- В C++11 також існують rvalue-посилання, таким чином, існує чотири комбінації спроб зробити посилання на посилання. Для їх колапсу (collapsing) маємо наступні правила:
- T&& && ⇒ T&&.
- T&& & ⇒ T&.
- T& && ⇒ T&.
- T& & ⇒ T&.
- Таким чином[9], lvalue-посилання завжди перемагає. І лише коли є rvalue-посилання на rvalue-посилання, результатом є rvalue-посилання.

[9] Нагадаю — при виведенні типів!

Forwarding reference (застаріла назва: універсальне посилання)

- Завдяки колапсу посилань, такий шаблон:

  ```
  template<typename T>
  void fn(T&& arg);
  ```

  якщо йому передати lvalue:

  ```
  int x;
  fn(x);
  ```

  дасть `fn<int&>(int& &&)` ⇒ `fn<int&>(int&)`, а:

  ```
  fn(11);
  fn( get_int_val() ); // int get_int_val() {....}
  ```

  безпосередньо: `fn<int>(int&&)`.
- Той же механізм працює і для auto:

  ```
  auto&& y1 = x; // y1 --> int&
  auto&& y2 = get_int_val(); // y2 --> int&&
  ```

- Скотт Меєрс пропонує такі конструкції називати універсальними посиланнями[10] — так як вони здатні прив'язуватися до чого завгодно. З різних причин, в стандарті таку сутність назвали forwarding reference.

[10] Текст від одного із захисників цього підходу: https://quuxplusone.github.io/blog/2022/02/02/look-what-they-need/.

- remove_reference<> в move(), власне, потрібен, щоб результат завжди був &&, навіть якщо передали & — запобігає колапсу.
- **Увага!** "Універсальність" стосується лише ситуацій, коли відбувається виведення типів — шаблонів і auto.
- Зокрема, в даному прикладі:

```
struct C1 {      .....  };

C1 fnC()  {  .....  }

void rvalref_fn(C1&& c) {          .....      }
```

такий код не скопілюється:

```
//C1 cx1;
//rvalref_fn(cx1); //! Error! "Cannot bind rvalue to &&"
```

а ось з таким все буде добре:

```
rvalref_fn(fnC());
```

- Перевантажувати функцію-шаблон, що отримує універсальне посилання, категорично не варто — воно майже завжди краще підходить, ніж альтернативи. Детальніше див. "Effective Modern C++" by Scott Meyers, Item 26.

- Зверніть **увагу**, ось тут:

```
template <class T>
class  Darr{          ....................
    void push_back(T&&){ ........ }
};
```

виведення типів, насправді, немає! Параметр шаблону є, звичайно, але він задається при інстанціюванні контейнера:

Darr<**double**> v;

і функція push_back фактично має прототип push_back(double&&).

- Універсальна поведінка має місце лише для прототипу виду T&&. Навіть "const T&&" "вбиває" таку поведінку, не говорячи вже про type<T>&&:

```
template<typename T>
void fn1(const T&& arg) {   .........   }

template<typename T>
void fn2(std :: vector<T>&& arg)  {  .........  }
```

В обох випадках аргумент поводитиметься як "звичайне" rvalue-посилання, і може прив'язуватися лише до rvalue.

const rvalue reference

- Константні rvalue-посилання позбавлені сенсу — переміщення, по своїй ідеї, має модифікувати аргумент.
- Однак, спеціалізоване використання йому придумали — забороняти виклик із rvalue.
- Приклад — оголошення ref()[11]:

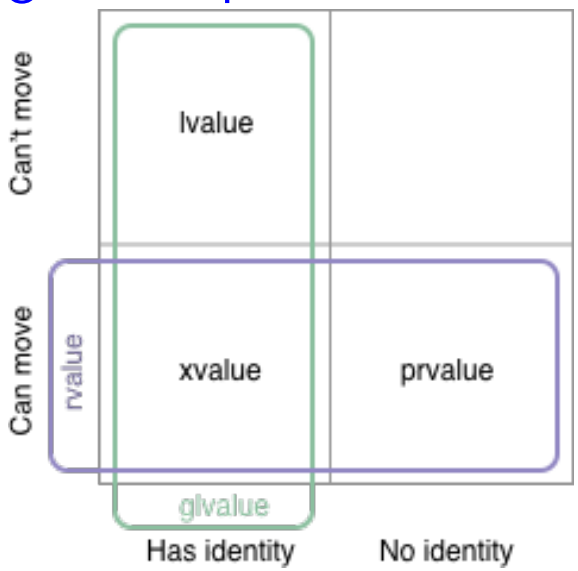
```
template< class T >
std :: reference_wrapper<T> ref(T& t);      // (1)
template< class T >
std :: reference_wrapper<T> ref( std::reference_wrapper<T> t );
                                            //(2)
template <class T>
void  ref (const T&&) = delete;                    // (3)
```

Варіант (3), власне, забороняє виклик цієї функції для rvalue.

[11] Для cref() — аналогічно.

Посилання

- Наочна ілюстрація суті різних типів значень: "lvalues, rvalues, glvalues, prvalues, xvalues, help!". Зображення звідти:



- "A prvalue is not a temporary" – есей про те, що prvalue не обов'язково є тимчасовим значенням, це може бути просто ідея об'єкту, яка використовується для конструювання абощо. (Але іноді вона може матеріалізуватися, наприклад, якщо код приймає таку ідею об'єкту за константним посиланням).

Реалізація переміщення

- Переміщаючи, можна "вкрасти" стан об'єкту – це важлива оптимізація, якщо він вже не буде потрібним в подальшому.
- Важлива вимога — об'єкт, із якого крадуть стан, слід залишити в коректному, консистентному, стані — для нього буде викликано деструктор. Крім того, формально, повинна бути можливість наново заповнити його даними, тому слід дотримати всіх його інваріантів.

- Найпростіший приклад — клас динамічного масиву, із вказівником на динамічний блок пам'яті та розміром:

```
template <class T>
class SimpleContainer {
    T* data;
    size_t size;
        ..............
}
```

- Тоді конструктор переміщення може мати вигляд:

```
SimpleContainer(SimpleContainer&& rhs) noexcept {
    data = rhs.data;
    size = rhs.size;
    rhs.data = nullptr;
    rhs.size = 0;
}
```

- Хоча мова цього не вимагає, дуже бажано конструктор переміщення робити noexcept – інакше, заради строгих гарантій щодо виключень, контейнери типу std::vector будуть песимізувати та виконувати копіювання. Детальніше див. https://quuxplusone.github.io/blog/2022/08/26/vector-pessimization/.

- А оператор присвоєння із переміщенням:

```
SimpleContainer& SimpleContainer::operator=(SimpleContainer&& rhs){
    if(&rhs != this){
        delete[] data;
        data = rhs.data;
        size = rhs.size;
        rhs.data = nullptr;
        rhs.size = 0;
    }
    return *this;
}
```

  Скористатися swap() у ньому – як у звичній ідіомі написання "звичайного" оператора присвоєння, `SimpleContainer& operator=(const SimpleContainer& rhs)`, буде не дуже хорошою ідеєю — агумент може бути xvalue, примусово отриманим із lvalue за допомогою move(), і тоді очистка відбудеться у неконтрольований автором коду момент.

- **Увага!** Переміщення в самого себе — невизначена поведінка, undefined behavior!
- Однак, цей клас, насправді, дуже простий — всі його члени є вбудованими типами.

- Розглянемо складніший приклад:

```
class M1{ ..... };
class M2{ ..... };
class M3{ ..... };
class Cm{
    M1 m1;
    M2 m2;
    M3 m3;
        .............
};
```

- Тоді реалізація конструктора переміщення виглядатиме так:

```
Cm(Cm&& rhs):
    m1(move(rhs.m1)), m2(move(rhs.m2)), m3(move(rhs.m3))  {}
```

- Справа в тому, що хоча аргумент і прийшов у функцію як rvalue-посилання, у ній він має ім'я – rhs, тому є lvalue! Тому, про переміщення мова не йтиме. Слід примусово викликати move().
- **Увага**, такий код: `m1(rhs.m1)` компілюватиметься і працюватиме, **однак**, копіюватиме, а не переміщатиме, аргументи!
- Зате, якщо об'єкт не підтримує переміщення, запит на нього є безпечним — просто відбудеться присвоєння.

- Зокрема, не зважаючи на всілякі move(), тут відбуватиметься копіювання:

```
class Cm{
    M1 m1;
        void set_m1(const M1& m): m1(move(m)){}
        //! Still copying!
        .............
};
```

Будьте обережні.

- Аналогічно, оператор присвоєння із переміщенням:

```
Cm& Cm::operator=(Cm&& rhs)
{
    if(&rhs != this)
    {
        m1 = move(rhs.m1);
        m2 = move(rhs.m2);
        m3 = move(rhs.m3);
    }
    return *this;
}
```

Переміщення з вбудованих типів

- Вбудовані типи просто копіюються під час переміщення.
- Базово це розумна поведінка, але іноді є бажання мати спеціальне значення "немає значення" – як 0 для вказівника, чи код помилки -1 для системних викликів POSIX.
- Це спеціальне значення має зберігатися в змінній при переміщенні з неї.
- Звичайно, компілятор сам такого не робитиме – але можна написати простий клас-обгортку.
- Тоді класи, які використовують її, простіше зможуть реалізувати zero-підхід – не писати явно операції присвоєння, копіювання, переміщення.
- Такі класи корисні і в рамках підходу строгих типів[12] – коли, скажімо, хоча фізично і маса і швидкість – це double, синтаксично це різні класи, і можна контролювати розмірність, помилки під час передачі аргументів тощо.

[12] Див., наприклад, https://www.fluentcpp.com/2016/12/08/strong-types-for-strong-interfaces/ та https://www.boost.org/doc/libs/1_81_0/doc/html/boost_units.html.

Переміщення з вбудованих типів

- Приклад з `https://devblogs.microsoft.com/oldnewthing/20230116-00/?p=107717` – "Making C++ primitive types meaningfully movable when they have sentinel values":

```
template<typename T, T empty_value>
struct movable_primitive {
  T value = empty_value;
  movable_primitive() = default;
  movable_primitive(T const& init) : value( init ) {}
  movable_primitive(movable_primitive const&) = default;
  movable_primitive(movable_primitive&& other) :
        value(std::exchange(other.value, empty_value)) {}
  movable_primitive& operator=(movable_primitive const&) = default;
  movable_primitive& operator=(movable_primitive&& other) {
        value = std::exchange(other.value, empty_value);
        return *this;
    }
  operator T() { return value; }
};
```

Неявно генеровані методи і переміщення

- Нагадаємо, що для класу, оголошеного так:

```
class A
{ };
```

компілятор створить:

```
class A{
    A();                                // Конструктор по замовчуванню
    ~A();                               // Деструктор по замовчуванню
    A(const A& rhs);                    // Конструктор копіювання
    A& operator=(const A& rhs); // Оператор присвоювання
    A(A&& rhs);                         // Конструктор переміщення
    A& operator=(A&& rhs); // Оператор присвоєння із переміщенням
    A* operator&();                     // Оператори отримання адреси
    const A* operator&() const;
};
```

- Оголошуючи їх самостійно пам'ятаємо про правило трьох/правило п'яти — якщо є нетривіальний конструктор копіювання, оператор присвоєння, деструктор, конструктор переміщення чи оператор присвоєння із переміщенням — майже завжди потрібні і всі решта!

Конструктор за замовчуванням та деструктор

- Конструктор по замовчуванню викликає конструктори базових класів, якщо такі є, для нестатичних членів-об'єктів викликає їх конструктори по замовчуванню залишає поля вбудованих типів неініціалізованими, (крім тих, які мають явні ініціалізатори).
- Якщо це неможливо – немає відповідних конструкторів[13], є члени-посилання чи члени-константи без ініціалізаторів, він не створюється.
- Якщо оголошено хоча б один користувацький конструктор, конструктор по замовчуванню також не створюється[14].
- Конструктор копіювання по замовчуванню поводиться аналогічно до конструктора по замовчуванню — копіює вбудовані типи побітово, решта – наскільки вдасться. (Див. вище про можливі ситуації, більш формально тут: `http://en.cppreference.com/w/cpp/language/copy_constructor`.)
- C++98 правила трьох не нав'язувала.
- C++11 намагається, наскільки це можливо, виправити ситуацію – не створювати автоматично майже гарантовано некоректні операції там, де це не заважає зворотній сумісності.

[13] Вони не оголошені, видалені або недоступні.

[14] Можна примусити генерувати за допомогою "default".

- Автоматично згенеровані конструктор переміщення та оператор присвоєння із переміщенням викликають переміщення для тих членів, які його підтримують, копіюють решту.
- Відповідно, вимоги до їх існування зрозумілі — щоб ці операції були можливими (наявність відповідних конструкторів чи операторів, також додатково вимагається наявність деструкторів).
- До C++11 move-операцій не було, тому їх поведінка не шкодить зворотній сумісності. Тому мова явно підтримує правило п'яти для них.
- Наявність деструктора вказує на існування неявних інваріантів. Тоді операції переміщення — конструктор переміщення та присвоєння із переміщенням, не створюються.
- Якщо в класі є нетривіальний конструктор копіювання чи оператор присвоєння — операції переміщення по замовчуванню не створюються. Це природно — вони є лише оптимізацією копіювання чи присвоєння.
- Аналогічно, створення хоча б однієї із операцій переміщення вимикає неявне створення операцій копіювання.

- У всіх випадках можна попросити компілятор все рівно згенерувати відповідні операції по замовчуванню, за допомогою default, але це рідко прийнятно.
- Також конструктор копіювання не створюється, якщо в класі є поля типу rvalue-посилань.
- Детальніше див., наприклад, `http://en.cppreference.com/w/cpp/language/move_assignment` і `http://en.cppreference.com/w/cpp/language/move_constructor` та Item 17 в "Effective Modern C++" Скота Меєрса.
- C++11 не могла заборонити автоматичне створення копіювання чи присвоєння, які порушується правило трьох — через зворотну сумісність. Однак, створення операцій присвоєння, якщо присутній явно заданий деструктор, автоматичне створення оператора присвоєння, якщо є конструктор копіювання і навпаки, визнано застарілими.
- **Увага!** Пам'ятаємо також, що шаблонні конструктори копіювання та оператори присвоєння не конкурують із не-шаблонними, навіть створеними по замовчуванню!

- Наявність деструктора забороняє автоматичне створення операцій переміщення. Однак, це адекватно не завжди.
- Важливий виняток — базовий клас повинен мати віртуальний деструктор, тому там він оголошується явно, навіть якщо є порожнім. Ще один, значно менш важливий, але іноді потрібний, випадок — відладочний код у деструкторі.
- Хороше рішення таке:

```
class Base {
public:
    virtual ~Base() = default;

    Base(Base&&) = default;
    Base& operator=(Base&&) = default;

    Base(const Base&) = default;
    Base& operator=(const Base&) = default;
```

- Взагалі, явно вказувати створення операцій переміщення по замовчуванню, **там де це прийнятно** — як у прикладах вище, має сенс навіть як оптимізація. Порушення умов створення операцій переміщення може привести до того, що код буде успішно компілюватися і працювати, лише жахливо повільно — копіюючи великі movable класи, замість того, щоб їх переміщати.

Інші використання переміщення. Perfect forwarding

- Існує цілий ряд класів, які копіювати не можна, а переміщати – без проблем. Приклад — std::unique_ptr та std::thread — копіювати їх позбавлено сенсу, але переміщати, наприклад, для повернення із функції — зручно.
- Можливість перевантажувати по тому, відбувається виклик із rvalue чи lvalue, може бути корисною не лише для конструкторів та присвоєнь.
- Наприклад:

```
struct D {
    .....
    map<string, string> data;

    D(const map<string, string>& d): data(d){}
    D(map<string, string>&& d): data(move(d)){}

    void setData(const map<string, string>& d) {
        data = d;
    }
    void setData(map<string, string>&& d) {
        data = move(d);
    }
```

- Однак, два варіанти кожного методу — це вже не зручно. При тому, що у них по одному аргументу. Для двох аргументів комбінацій буде вже чотири! І т.д. — код виходить важким у підтримці.
- Хотілося б, щоб prvalue та xvalue переміщалися, а lvalue копіювалися автоматично.
- Такого можна добитися, скориставшись perfect forwarding (ідеальне перенаправлення?) та forwarding reference:

```
struct E0 {
    map<string, string> data1;
    vector<deque<string>> data2;

    template<typename T1, typename T2>
    E0(T1&& d1, T2&& d2):
        data1(std::forward<T1>(d1)), data2(std::forward<T2>(d2))
    {}

    template<typename T1, typename T2>
    void setData(T1&& d1, T2&& d2) {
        data1 = std::forward<T1>(d1);
        data2 = std::forward<T2>(d2);
    }
};
```

- Наступна конструкція прийме аргументи будь якого типу, завдяки правилу колапсу посилань:

```
template<typename T1, typename T2>
void setData(T1&& d1, T2&& d2) { ..... }
```

  - rvalue буде передано як rvalue-посилання,
  - rvalue-посилання буде передано як rvalue-посилання,
  - константне lvalue буде передано як константне посилання на lvalue,
  - lvalue буде передано як посилання на lvalue,
  - lvalue-посилання буде передано як lvalue-посилання.
- Тепер слід вказати переміщати rvalue-посилання і копіювати все решта. move() тут не підходить — вона вказує виконати безумовне переміщення.
- Допоможе функція forward(). Вона, як і move(), яка нічого не переміщає, нічого не форвардить, лише робить належне приведення типів – до rvalue reference, лише там де отримано rvalue.
- Тому слід застосовувати move() до rvalue reference і forward() до forwarding reference.

forward<>()

- Ця функція користується можливістю перевантаження між lvalue-посиланнями та rvalue-посиланнями. Її оголошення та можлива реалізація[15]:

```
template< class T >
T&& forward( typename std::remove_reference<T>::type& t ){
    return static_cast<T&&>(t);
}
template< class T >
T&& forward( typename std::remove_reference<T>::type&& t ){
    static_assert (!std :: is_lvalue_reference<T>::value,
        "template argument is an lvalue reference type");
    return static_cast<T&&>(t);
}
```

- Реалізація забороняє автоматичне виведення типів[16], щоб заборонити виклики виду forward(string{}) — вони завжди даватимуть lvalue, так як параметр має ім'я. Тип слід вказувати явно: forward<string>(s).
- static_assert страхує від ситуації, коли надміру хитрий програміст використав явне вказання типу forward<string&>(string{}).

[15] Взято з незначними модифікаціями із GCC-5.2. C++14 додав constexpr до оголошення.

[16] Завдяки remove_reference<>, використання якого, крім "здирання" посилань, робить T таким, що не виводиться — "non-deduced context".

Perfect forwarding

- Така можливість передачі параметрів без втрати інформації про тип, константність і "rvalueness" чи "lvalueness" називається perfect forwarding.
- Працює лише із шаблонами!
- Надзвичайно корисна можливість:
- Дозволяє не писати всі можливі комбінації функцій із багатьма аргументами, частину із яких може бути потреба переміщати.
- Можна створювати функції типу emplace_back() чи конструктора класу thread, які точно передають аргументи відповідним конструкторам чи Callable, коректно вибираючи правильне перевантаження. (Можливо, із допомогою ref() чи cref() там, де слід передавати аргумент за посиланням, а не як lvalue.)
- Тобто, простіше писати ефективний і лаконічний код.
- Особливо корисною ця можливість є із variadic templates — шаблонами із змінною кількістю аргументів.
- Однак, пам'ятайте – компілятор не володіє телепатією: "Perfect forwarding forwards objects, not braced things that are trying to become objects".
- Див. також: "Perfect forwarding and universal references in C++".

Функції типу std::forward і продуктивність Debug-компіляцій

- Такі допоміжні функції, однак, не дуже добре опрацьовуються сучасними компіляторами в режимі без оптимізації (Debug).
- Вони – чисто синтаксичний цукор, однак без оптимізації компілятори вставляють їх виклики: `https://vittorioromeo.info/index/blog/debug_performance_cpp.html`.
- Хоча, на початок 2023 ситуація починає покращуватися: `https://devblogs.microsoft.com/cppblog/improving-the-state-of-debug-performance-in-c/` для MSVC, `https://quuxplusone.github.io/blog/2022/12/24/builtin-std-forward/` – Clang.

RVO і move()

- Move() може інтерферувати із RVO (Return value optimization)/Named Return Value Optimization (NRVO)/copy elision, тому не варто робити так:

```
C fn() {
        C res;
        ....
        return move(res); //! Погана ідея!
                // Може заборонити компілятору
                // зробити RVO, без жодного виграшу
}
```

- Справа в тому, що RVO дозволене для локальних об'єктів (в тому числі, тимчасових, обчислених у виразі return), якщо тип точно співпадає із типом аргументу.
- Але навіть якщо компілятор вирішив не скористатися RVO (заборонено, починаючи із C++17), результат буде rvalue reference і move() все рівно не потрібен.
- Увага! Аргументи функції не вважаються локальними об'єктами – їх це не стосуватиметься.
- Детальніше див. "cppreference: Copy elision" та "Effective Modern C++", главу 25.

RVO і move() – 2

- Однак, іноді move() таки потрібен для RVO.
- RVO працює завдяки тому, що об'єкт створюється зразу у стеку caller. Іноді це важко організувати:

```
C fn(bool arg){
    C x, y;
    if (arg) {
        return x;
    }
    return y;
}
```

Тоді вимушено (неявно) буде зроблено move().

- Але якщо замість if() використати тернарний оператор – move слід робити явно:

```
C fn(bool arg){
    C x, y;
        return std::move(arg ? x : y);
}
```

RVO і move() – 3

- Якщо ж тип різний – move() слід використовувати явно:

```
class B{};
class D: public B{};

B fn(bool arg){
    D x, y;
    if (arg) {
        return std::move(x);
    }
    return std::move(y);
}
```

- Clang має відповідне попередження – "а чи не забули ви явний move?": -Wreturn-std-move.

RVO і move() – 4

- Також, щоб RVO могла спрацювати, потрібна наявність конструктора копіювання або конструктора переміщення – навіть якщо їх ніколи не буде викликати: "The move constructor that you have to declare, even though you don't want anyone to actually call it".
- "On harmful overuse of std::move", про шкідливість "return std::move(var)" та "std::string name = std::move(std::to_string(id));" – тут move заважатиме RVO.
- "More on harmful overuse of std::move".
- Суміжна тема – до C++23, такий код не компілювався:

```
Widget&& four(Widget&& w) {
    return w;  // OK в C++23+
}
```

Недосконалості perfect forwarding

- Є ситуації, коли forwarding не такий вже й perfect.
- 0 не буде передано як нульовий вказівник. Користуйтеся nullptr.
- Списки ініціалізації — initializer_list ніколи не виводиться як тип-аргумент шаблону. (Однак, змінна типу initializer_list "відфорвардиться" успішно!):

```
template<typename Ct, typename... Types>
void forward_all(Ct cfn, Types&&... args){
      cfn(std::forward<Types>(args)...);
}
void fn(const vector<int>& v ){.....}

    fn ({1,2,3}); // OK
    // forward_all(fn, {1,2,3}); // Error!
    auto tv = {1,2,3};
    forward_all(fn, tv);      // OK
```

- Статичні константи інтегральних типів, оголошені безпосередньо в тілі класів можуть приводити до помилок часу лінкування[17]. Вирішується їх явним визначенням в cpp-файлі. Справа в тому, що такі константи, в нормі, не існують як об'єкт взагалі – constant propagation їх позбувається. Однак, під час форвардингу компілятор маніпулює посиланнями (які часто реалізуються вказівниками), а взяття адрес таких констант таки потребуватиме окремого визначення.
- Якщо функція приймає вказівник на функцію, і їй передати ім'я перевантаженої функції, компілятор вибере правильний виклик. Однак, perfect forwarding такої можливості не матиме, тому вказівники на перевантажені функції не можна форвардити. На жаль, це стосується, в тому числі, маніпуляторів потоків вводу-виводу. Те ж стосується шаблонів функцій — автоматичного виведення типів немає.
- Бітові поля не форвардяться, так як заборонено створювати не-константні посилання на них, а весь механізм perfect forwarding крутиться навколо посилань.

[17] Але можуть і не призводити – бо посилання не завжди реалізовуються вказівниками. Однак, покладатися на це – непортабельно.

Посилання

- Див. також ref() і cref():

```
voif  fn(double& res);

double put_here;
thread  t(fn,  ref(put_here));
```

Детальніше про perfect forwarding див.:

- `http://eli.thegreenplace.net/2014/perfect-forwarding-and-universal-references-in-c/` — Perfect forwarding and universal references in C++.
- "Effective Modern C++" by Scott Meyers, Item 30.
- `http://en.cppreference.com/w/cpp/utility/forward`.
- `http://stackoverflow.com/questions/27501400/the-implementation-of-stdforward`
- `http://stackoverflow.com/questions/8526598/how-does-stdforward-work`.
- `http://stackoverflow.com/questions/10335916/c11-why-is-static-assert-in-stdforward-necessary`.

std::move_if_noexcept

- std::move_if_noexcept() — цікава функція, яка повертає rvalue-посилання, якщо конструктор переміщення не кидає виключень, lvalue-посилання в іншому випадку[18]:

```
template< class T >
typename std::conditional<
        !std::is_nothrow_move_constructible<T>::value &&
         std::is_copy_constructible<T>::value,
        const T&,
        T&&
>::type move_if_noexcept(T& x);
```

- Можливе використання — комбінування переміщення та строгих гарантій щодо виключень. Див. також `http://en.cppreference.com/w/cpp/utility/move_if_noexcept`.

[18] C++14 додав constexpr до оголошення.

Алгоритм переміщення

- Алгоритм std::move():

```
template< class InputIt, class OutputIt >
  OutputIt move( InputIt first, InputIt last, OutputIt d_first );
```

переміщає елементи із одного інтервалу в інший, користуючись функцією move(). Повертає ітератор на перший не переміщений елемент у цільовому інтервалі: d_first + (last - first).

- Переміщає зліва направо.
- Алгоритм std::move_backward():

```
template< class BidirIt1, class BidirIt2 >
  BidirIt2 move_backward( BidirIt1 first, BidirIt1 last,
                          BidirIt2 d_last );
```

робить те ж, справа наліво.

Шпаргалка

- Трапилися мені тут: `https://blog.feabhas.com/2018/04/handy-cut-keep-guide-stdforward-stdmove/` дуже наочні таблички (приклади див. за посиланням!).
- Якщо ви пишете функцію:

The parameter is...	The function is... not a template	The function is... a template
T&	-	-
T&&	`std::move`	`std::forward<T>`

- Якщо ви викликаєте функцію:

After the call...	The expression is... l-value	The expression is... r-value
I need the object	-	-
I don't need the object	`std::move`	-

Стан об'єктів, із яких перемістили

- Об'єкт, із якого перемістили, має перебувати в коректному, хоч і не визначеному стані. Тобто, в ідеалі, із таким об'єктом можна виконувати будь-які операції – його інваріанти мають дотримуватися.
- Стандартна бібліотека, як мінімум – для більшості класів, це гарантує.
- Саттер наголошує, що клас, який порушує таку вимогу – помилковий: `http://herbsutter.com/2020/02/17/move-simply/`.
- Однак, так як деякі автори "зрізають кути", для оптимізації абощо, на практиці, робити із класом щось крім знищення чи присвоєння може бути небезпечним: `https://cor3ntin.github.io/posts/move/`. Звіряйтеся із документацією.

Посилання

- https://devblogs.microsoft.com/oldnewthing/20221123-00/?p=107443 – "Sometimes perfect forwarding can be too perfect: Lazy conversion is lazy". Приклад, коли використання std::make_unique для класу із приватним конструктором предка, спричиняє проблеми. Ситуація, звичайно, екзотична – але ілюструє механізми.
- https://quuxplusone.github.io/blog/2023/05/27/dont-forward-non-forwarding-references/ – не використовуйте std::forward там, де не відбувається форвардинг. Для вираження ідеї "зберегти rvalue-referenceness" скористайтеся static_cast<T&&>(...).
- https://cppdepend.com/blog/want-to-know-where-the-c-move-feature-is-used-on-your-behalf/ – як, за допомогою "clang -cc1 -ast-dump" подивитися, які виклики генерує компілятор, зокрема – коли він викликає надані чи створені ним конструктори переміщення.
- Reader Q&A: What's the best way to pass an istream parameter? – зокрема, аналізується випадок потреби передавати як lvalue, так і rvalue та як це робити акуратно.

Deducing this – C++23

- Чотири варіанти перевантажень, в залежності від категорії значення (&, const &, &&, const &&), можуть бути проблемою.
- Часто вони роблять одне і те ж, і доводиться або дублювати реалізацію чотири рази, або реалізовувати один раз і делегувати з використанням static_cast до відповідних варіантів, або доручати це допоміжній функції – з тими ж перетвореннями типів. Або, замість методів використати незалежну функцію з forwarding reference[19] – але це змінює інтерфейс.
- Доповнення Deducing this дозволяє використати другий підхід і для методів.
- Для цього воно дозволяє явно вказувати об'єкт, який використовуватиме метод[20]:

```
class explicit_this {
    int data_m = 0;
    ...............
    void method(this explicit_this & self) {
        auto t = self.data_m;
            ...........
    }
};
```

Тоді явне чи неявне використання this для звертання до членів заборонене.

- Такі методи не можуть бути статичними, віртуальними чи з const.
- Ім'я self не є зафіксованим, його використання – лише домовленість.

[19] Див. презентацію про lvalue refences та perfect forwarding.

[20] Аналогічно до явного self в Python та деяких інших мовах.

- Тоді, наприклад, замість таких чотирьох перевантажень з практично тотожним кодом:

```
template <typename T>
class optional {
    ....................
  constexpr T&          value() &          { ..... }
  constexpr T const&    value() const&     { ..... }
  constexpr T&&         value() &&         { ...<uses std::move>... }
  constexpr T const&&   value() const&&    { ...<uses std::move>... }
};
```

використовуємо лише одне:

```
template <typename T>
class optional {
    ....................
  template <class Self>
    constexpr auto&& value(this Self&& self) { ...<uses std::forward>... }
};
```

- Важливо, що поведінка цієї конструкції передбачувана – шаблон як шаблон, зі всіма його правилами і без додаткових.
- Більш розгорнутий приклад наведено в оригінальному документі-пропозиції: "Deducing this " та пості від одного з його авторів: "C++23's Deducing this: what it is, why it is, how to use it".
- Однак, цей інструмент корисний і для інших задач.

Спрощення Curiously Recurring Template Pattern (CRTP)

- Без, на прикладі "mixin" add_postfix_increment[21]:

```
template <typename Derived>
struct add_postfix_increment {
    Derived operator++(int) {
        auto& self = static_cast<Derived&>(*this);

        Derived tmp(self);
        ++self;
        return tmp;
    }
};
struct some_type : add_postfix_increment<some_type> {
    // Префіксний ++, який буде використано add_postfix_increment
    some_type& operator++();
};
```

- З deducing this – синтаксис помітно простіший:

```
struct add_postfix_increment {
    template <typename Self>
    auto operator++(this Self&& self, int) {
        auto tmp = self;
        ++self;
        return tmp;
    }
};
```

Forward з лямбда-функціями та std::forward_like

- Приклад з документу, що запропонував std::forward_like – вибір, як виконувати форвардинг в залежності від категорії замикання:

```
auto callback = [m=get_message(), &scheduler](this auto &&self) -> bool {
    return scheduler.submit(std::forward_like<decltype(self)>(m));
};
```

- std::forward_like – вибирає категорію одного об'єкта в залежності від категорії іншого. При цьому константності об'єктів об'єднуються.
  - Зокрема, це може бути різний форвардинг члену об'єкта, в залежності від категорії самого об'єкту.
- Див. також: Daily bit(e) of C++ | std::forward_like.

Рекурсивні лямбда-функції

- Завдяки використанню deducing this з лямбда-функціями, простіше створювати рекурсивні лямбди:

```
auto fib = [](this auto self, int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};
```

- Вони можуть бути корисними з патерном visitor, зокрема і std::visit для std::variant.
- Ще один приклад – див. тут.

Передавання this за значенням

- Оскільки self/this передається явно, можна передати його за значенням.

```
class some_small_t {
    ..................
     int uwu(this some_small_t self);
};
```

- Для малих об'єктів це може бути ефективніше!

